METROPOLIA UNIVERSITY OF APPLIED SCIENCES
DEPARTMENT OF INFORMATION & COMMUNICATION TECHNOLOGY

FINAL SEMESTER EXAMINATION: C# & .NET FRAMEWORK DEVELOPMENT

Course Code: CS-NET301 | Academic Year: 2026 | Duration: 3 Hours | Total Marks: 100

Student Name: _____	**Student ID:** _____
Degree Program: Software Engineering	**Date:** October 13, 2026
Instructions: Answer ALL 100 questions. Write clear code snippet solutions where required. No external notes allowed.	**Total Pages:** 6

PART I: MULTIPLE CHOICE QUESTIONS (30 Marks — 1 Mark Each)

Select the single best answer for each question.

Q 1. Which component of the .NET Framework manages memory allocation, thread execution, and code safety?

A) Common Language Specification (CLS) **B) Common Language Runtime (CLR)**
C) Just-In-Time Compiler (JIT) **D) Microsoft Intermediate Language (MSIL)**

Q 2. What is the primary role of the JIT (Just-In-Time) compiler in the .NET compilation process?

A) Converts C# source code directly to native machine code **B) Converts C# source code to MSIL bytecode**
C) Converts MSIL bytecode into CPU-specific native machine instructions at runtime **D) Manages garbage collection cycles**

Q 3. Which data type in C# is a value type and stored on the Stack by default?

A) System.String **B) System.Array**
C) System.Int32 **D) Custom Class object**

Q 4. What is the output of 'Console.WriteLine(15 / 4);' in C#?

A) 3.75 **B) 3**
C) 4 **D) 3.8**

Q 5. What happens when you explicitly cast a double value of 9.99 to an integer using '(int)9.99'?

A) Rounds to 10 **B) Truncates to 9**
C) Throws OverflowException **D) Results in a compilation error**

Q 6. Which keyword is used in C# to prevent a class from being inherited by other classes?

A) static B) readonly
C) sealed D) abstract

Q 7. What is the key difference between 'ref' and 'out' parameter modifiers in C#?

A) 'ref' requires initialization before passing; 'out' must be assigned inside the method. B) 'out' requires initialization before passing; 'ref' must be assigned inside.
C) 'ref' works only with value types; 'out' works only with reference types. D) 'out' creates a copy; 'ref' passes by memory address.

Q 8. Which interface must be implemented to support 'foreach' iteration over a custom collection?

A) IComparable B) IEnumerable
C) IDisposable D) ICloneable

Q 9. What is the correct syntax to declare and instantiate a dynamic List of strings in C#?

A) string[] list = new string[]; B) List<string> list = new List<string>();
C) ArrayList<string> list = new ArrayList(); D) List list = new string List();

Q 10. In C#, what exception is thrown when attempting to access an element at index 5 in an array of size 5?

A) NullReferenceException B) IndexOutOfRangeException
C) ArgumentOutOfRangeException D) InvalidOperationException

Q 11. How are C# strings treated in memory regarding mutability?

A) Strings are mutable value types stored on the stack. B) Strings are immutable reference types; modifying a string creates a new instance on the heap.
C) Strings are stored in static read-only registers. D) Strings automatically resize without heap allocation.

Q 12. Which statement accurately describes Garbage Collection (GC) in .NET?

A) GC runs deterministically every 10 seconds. B) GC reclaims memory allocated on the Heap for unreachable objects automatically.
C) GC reclaims Stack memory when methods return. D) GC requires explicit invocation via 'delete' keyword.

Q 13. What is the effect of the 'using' statement when managing unmanaged resources (e.g., StreamReader)?

A) Keeps the file open permanently. B) Ensures Dispose() is called automatically even if exceptions occur.
C) Converts the file into a JSON string automatically. D) Suppresses all runtime exceptions.

Q14. Which operator is used for null-coalescing in C# (e.g., 'var x = name ?? "Guest";')?

A) ? **B) ?.**
C) ?? **D) ::**

Q15. What is the output of 'string.Format("{0:C}", 25)' assuming US locale?

A) 25.00 **B) $25.00**
C) 25% **D) 0025**

Q16. Which access modifier allows a class member to be accessed only within its containing class or derived classes?

A) private **B) internal**
C) protected **D) protected internal**

Q17. What is boxing in C#?

A) Converting a reference type to a value type. **B) Converting a value type to an object reference type on the heap.**
C) Wrapping a class in a namespace. **D) Compiling C# code into MSIL.**

Q18. What is unboxing in C#?

A) Explicitly extracting a value type from an object instance on the heap. **B) Implicitly converting an int to double.**
C) Deleting a file from disk. **D) Freeing stack frame parameters.**

Q19. Which control structure is guaranteed to execute its code body at least once?

A) for loop **B) while loop**
C) do-while loop **D) foreach loop**

Q20. What keyword is used to explicitly call a base class constructor from a derived class constructor?

A) this **B) base**
C) super **D) parent**

Q21. Which method of the System.IO.File class reads all lines of a text file into a string array?

A) File.ReadText() **B) File.ReadAllLines()**
C) File.OpenStream() **D) File.LoadArray()**

Q 22. What is the return type of a method declared as 'static void Display(string msg)'?

A) string **B) int**
C) None (void) **D) bool**

Q 23. Which keyword allows modifying a method in a derived class to replace a base class method marked 'virtual'?

A) new **B) override**
C) extends **D) implements**

Q 24. What happens when 'int.TryParse("ABC", out int result)' is executed?

A) Throws FormatException **B) Returns false and sets result to 0**
C) Returns true and sets result to -1 **D) Throws InvalidCastException**

Q 25. Which class in System.Collections.Generic implements a First-In, First-Out (FIFO) collection?

A) Stack<T> **B) Queue<T>**
C) Dictionary<TKey, TValue> **D) HashSet<T>**

Q 26. Which collection type stores key-value pairs with O(1) average lookup speed?

A) List<T> **B) LinkedList<T>**
C) Dictionary<TKey, TValue> **D) Array**

Q 27. What does the 'const' keyword signify when applied to a field?

A) The value is determined at runtime and cannot change. **B) The value is evaluated at compile-time and cannot be modified.**
C) The field can only be assigned inside the constructor. **D) The field is stored on the heap.**

Q 28. What does the 'readonly' keyword allow that 'const' does NOT allow?

A) Initialization at compile-time only. **B) Initialization inside the class constructor at runtime.**
C) Modification anytime in public methods. **D) Conversion to value types.**

Q 29. Which namespace contains classes like File, Directory, StreamReader, and StreamWriter?

A) System.Text **B) System.Collections**
C) System.IO **D) System.Threading**

Q30. What is the entry point method signature for a standard C# Console Application?

A) public void Main() **B) static void Main(string[] args)**
C) static int Start() **D) public static void Run()**

PART II: SHORT ANSWER & CONCEPTUAL ESSAY QUESTIONS (20 Marks — 1 Mark Each)

Provide concise, accurate technical responses for each item.

Q31. Explain the distinction between MSIL (Intermediate Language) and native machine code in .NET.

Answer: _____

Q32. Describe how Stack memory differs from Heap memory in terms of allocation, speed, and lifetime.

Answer: _____

Q33. Differentiate between Value Types and Reference Types. Provide two C# code examples of each.

Answer: _____

Q34. Explain how short-circuit evaluation works with C# logical operators '&&' and '||'.

Answer: _____

Q35. What is method overloading? State the rules regarding parameter signatures.

Answer: _____

Q36. What is method overriding? How do 'virtual' and 'override' keywords facilitate polymorphism?

Answer: _____

_____

Q 37. Explain the concept of Encapsulation and why backing fields are kept private with public properties.

Answer: _____

_____

Q 38. Compare 'Array' vs 'List<T>' in terms of size flexibility, performance, and memory overhead.

Answer: _____

_____

Q 39. Explain why string concatenation in a large loop using '+' creates performance bottlenecks and how 'StringBuilder' solves it.

Answer: _____

_____

Q 40. What is the purpose of the 'try-catch-finally' block? When does 'finally' execute?

Answer: _____

_____

Q 41. Explain the difference between throwing an exception with 'throw;' vs 'throw ex;' inside a catch block.

Answer: _____

_____

Q 42. What are C# Properties? Differentiate auto-implemented properties from full properties with backing fields.

Answer: _____

_____

Q 43. Explain the role of namespaces in C# projects and how the 'using' directive prevents naming collisions.

Answer: _____

_____

Q 44. Describe the execution flow when an unhandled exception occurs in a nested method call.

Answer: _____

_____

Q 45. What is an Abstract Class? How does it differ from an Interface?

Answer: _____

_____

Q 46. Explain Boxing and Unboxing overhead and why generic collections like List<int> avoid it.

Answer: _____

_____

Q 47. What is the Garbage Collector's role regarding Generation 0, 1, and 2 heaps in .NET?

Answer: _____

_____

Q 48. Explain the purpose of 'Console.ReadLine()' when parsing user numerical input and how 'double.TryParse' protects against crashes.

Answer: _____

_____

Q 49. Describe the difference between '==' equality comparison and '.Equals()' for reference objects.

Answer: _____

_____

Q 50. Explain how file streams (StreamReader/StreamWriter) must be managed to prevent resource locks on operating systems.

Answer: _____

_____

PART III: CODE ANALYSIS, DEBUGGING & IMPLEMENTATION (30 Marks — 1 Mark Each)

Write complete C# code snippets or analyze the provided code blocks.

Q 51. Write a C# statement to prompt the user for their age and parse it safely into an integer variable 'age' without crashing.

Sample Solution:
```
int age; Console.Write("Enter age: "); bool valid = int.TryParse(Console.ReadLine(), out age);
```

Q 52. Write a C# method 'bool IsEven(int number)' that returns true if a number is even, false otherwise.

Sample Solution:
```
static bool IsEven(int number) => number % 2 == 0;
```

Q 53. Write a C# 'for' loop that prints all odd numbers between 1 and 20 (inclusive) on a single line.

Sample Solution:
```
for (int i = 1; i <= 20; i += 2) { Console.Write(i + " "); }
```

Q 54. Given int[] arr = { 10, 20, 30, 40, 50 }; write a foreach loop to calculate and print the sum of elements.

Sample Solution:
```
int sum = 0; foreach (int num in arr) sum += num; Console.WriteLine("Sum: " + sum);
```

Q 55. Write code to instantiate a List<string> named 'cities', add three city names, and remove the second city.

Sample Solution:
```
List<string> cities = new List<string>() { "Helsinki", "Espoo", "Vantaa" };
cities.RemoveAt(1);
```

Q 56. Analyze this code: 'int a = 5; int b = 0; int c = a / b;' What runtime exception occurs?

Sample Solution:
```
System.DivideByZeroException is thrown at runtime.
```

Q 57. Rewrite the following if-else structure into a cleaner C# switch statement: if (day == 1) Console.WriteLine("Mon"); else if (day == 2) Console.WriteLine("Tue"); else Console.WriteLine("Other");

Sample Solution:
```
switch(day) { case 1: Console.WriteLine("Mon"); break; case 2: Console.WriteLine("Tue"); break; default: Console.WriteLine("Other"); break; }
```

Q 58. Write a C# method 'int FindMax(int[] numbers)' that returns the largest integer in an array.

Sample Solution:
```
static int FindMax(int[] numbers) { int max = numbers[0]; foreach(int n in numbers) if (n > max) max = n; return max; }
```

Q 59. Write code using 'StreamWriter' to write the line "Learning .NET Framework" into a file named "log.txt".

Sample Solution:
```
using (StreamWriter sw = new StreamWriter("log.txt", true)) { sw.WriteLine("Learning .NET Framework"); }
```

Q 60. Write code using 'StreamReader' to read and display the first line from "data.txt".

Sample Solution:
```
using (StreamReader sr = new StreamReader("data.txt")) { string line = sr.ReadLine(); Console.WriteLine(line); }
```

Q 61. Identify the logical error: 'double avg = (80 + 90 + 85) / 3;' How do you fix it to get precise decimal division?

Sample Solution:
```
Integer division drops precision. Fix: 'double avg = (80 + 90 + 85) / 3.0;'
```

Q 62. Write a C# constructor for a class 'Book' that initializes private fields 'title' and 'author'.

Sample Solution:
```
public Book(string t, string a) { title = t; author = a; }
```

Q 63. Write a full C# auto-property for 'Price' with a public getter and private setter.

Sample Solution:
```
public double Price { get; private set; }
```

Q 64. Write a C# program snippet that counts how many times the character 'a' appears in a string 'text'.

Sample Solution:
```
int count = 0; foreach (char c in text) if (c == 'a' || c == 'A') count++;
```

Q 65. What is the result of 'string s = null; Console.WriteLine(s?.Length ?? 0);'?

Sample Solution:
```
Prints 0 without throwing NullReferenceException.
```

Q 66. Write a C# method that accepts a double 'price' and applies a 15% discount, returning the final price.

Sample Solution:
```
static double Discount(double price) => price * 0.85;
```

Q 67. Write code to reverse an array 'int[] values = { 1, 2, 3, 4, 5 };' in-place or using System.Array.

Sample Solution:
```
Array.Reverse(values);
```

Q 68. Write a C# try-catch block that catches FormatException when converting string to int.

Sample Solution:
```
try { int val = int.Parse(input); } catch(FormatException) { Console.WriteLine("Invalid integer format."); }
```

Q 69. Define a C# enum named 'Grade' with values A, B, C, D, F.

Sample Solution:
```
enum Grade { A, B, C, D, F }
```

Q 70. Write a C# method 'static void Swap(ref int x, ref int y)' that swaps two integer values.

Sample Solution:
```
static void Swap(ref int x, ref int y) { int temp = x; x = y; y = temp; }
```

Q 71. Write code to check if a key "Admin" exists in a Dictionary<string, string> 'roles'.

Sample Solution:
```
if (roles.ContainsKey("Admin")) { /* Key exists */ }
```

Q 72. Write a do-while loop that keeps prompting user for input until they enter "exit".

Sample Solution:
```
string input; do { input = Console.ReadLine(); } while (input != "exit");
```

Q 73. Write a C# extension method or method 'CountWords(string sentence)' that returns word count split by spaces.

Sample Solution:
```
static int CountWords(string sentence) => sentence.Split(' ', StringSplitOptions.RemoveEmptyEntries).Length;
```

Q 74. Convert string "123.45" to a double variable using invariant culture or standard double.Parse.

Sample Solution:
```
double num = double.Parse("123.45", System.Globalization.CultureInfo.InvariantCulture);
```

Q 75. Write code to join a string list List<string> tags = new List<string>{"C#", ".NET", "OOP"} with commas.

Sample Solution:
```
string result = string.Join(", ", tags);
```

Q 76. Create an array of 5 Student objects: 'Student[] students = new Student[5];' How do you initialize element 0?

Sample Solution:
```
students[0] = new Student();
```

Q 77. Write a method that takes an array of integers and returns a List<int> containing only positive numbers.

Sample Solution:
```
static List<int> GetPositives(int[] nums) { List<int> res = new List<int>(); foreach(int n in nums) if(n > 0) res.Add(n); return res; }
```

Q 78. Write code using StringBuilder to append 10 numbers from 1 to 10 separated by hyphens.

Sample Solution:
```
StringBuilder sb = new StringBuilder(); for(int i=1; i<=10; i++) sb.Append(i).Append(i < 10 ? "-" : "");
```

Q 79. Write a method signature that uses 'params' keyword to accept a variable number of integer arguments.

Sample Solution:
```
static int SumAll(params int[] numbers) { int s = 0; foreach(int n in numbers) s+=n; return s; }
```

Q 80. Write a C# statement to round a double 'val = 3.14159' to 2 decimal places.

Sample Solution:
```
double rounded = Math.Round(val, 2);
```

PART IV: CODE CORRECTION & BUG FIXING (20 Marks — 1 Mark Each)

Identify the bug in each buggy snippet and write the corrected code.

Q 81. BUGGY: int[] arr = new int[3]; arr[3] = 10;

Correction: Out of bounds index (valid indices 0..2). FIX: arr[2] = 10;

Q 82. BUGGY: string name; Console.WriteLine(name.Length);

Correction: Unassigned local variable. FIX: string name = ""; Console.WriteLine(name.Length);

Q 83. BUGGY: public class Car { public const double Speed; }

Correction: Const fields must be initialized at declaration. FIX: public const double Speed = 60.0;

Q 84. BUGGY: List<int> nums = null; nums.Add(5);

Correction: NullReferenceException. FIX: List<int> nums = new List<int>(); nums.Add(5);

Q 85. BUGGY: static void Main() { int x = 10; Change(x); } static void Change(int x) { x = 20; }

Correction: Passed by value, x remains 10. FIX: Use 'ref' modifier: static void Change(ref int x) { x = 20; }

Q 86. BUGGY: if (x = 5) { Console.WriteLine("Five"); }

Correction: Assignment '=' used instead of comparison '=='. FIX: if (x == 5)

Q 87. BUGGY: float f = 3.14;

Correction: Literal 3.14 is double. FIX: float f = 3.14f;

Q 88. BUGGY: public class Person { private string name; } Person p = new Person(); p.name = "Alice";

Correction: 'name' is private and inaccessible. FIX: Make property public or provide public setter.

Q 89. BUGGY: string s1 = "hello"; string s2 = "hello"; if (s1 = s2)

Correction: Syntax error in condition. FIX: if (s1 == s2)

Q 90. BUGGY: for (int i = 0; i <= arr.Length; i++) { Console.WriteLine(arr[i]); }

Correction: Loop condition '<=' causes IndexOutOfRangeException. FIX: for (int i = 0; i < arr.Length; i++)

Q 91. BUGGY: StreamReader sr = new StreamReader("file.txt"); string line = sr.ReadLine();

Correction: Stream is not closed, leaving resource locked. FIX: Wrap in 'using (StreamReader sr = ...)'

Q92. BUGGY: switch(x) { case 1: Console.WriteLine("One"); case 2: Console.WriteLine("Two"); }

Correction: Missing break statements cause compile failure. FIX: Add 'break;' to each case.

Q93. BUGGY: int res = int.Parse(Console.ReadLine()); // when user enters "abc"

Correction: Crashes with FormatException. FIX: Use int.TryParse().

Q94. BUGGY: public class Circle { public abstract void Draw(); }

Correction: Class must be marked abstract if it contains abstract methods. FIX: public abstract class Circle

Q95. BUGGY: double x = 5/2;

Correction: Integer division yields 2.0. FIX: double x = 5.0 / 2;

Q96. BUGGY: public void Process() { try { File.ReadAllText("a.txt"); } catch { throw ex; } }

Correction: Destroys original stack trace. FIX: Use 'throw;'

Q97. BUGGY: int[] nums = { 1, 2, 3 }; nums[1] = null;

Correction: Cannot assign null to value type int. FIX: Use int?[] or assign numeric value.

Q98. BUGGY: string str = "Test"; str[0] = 't';

Correction: Strings are immutable; indexer is read-only. FIX: string str = "t" + str.Substring(1);

Q99. BUGGY: class Base { public void Show(){} } class Derived : Base { public override void Show(){} }

Correction: Base method not marked virtual. FIX: Mark Base.Show() as 'public virtual void Show()'

Q100. BUGGY: static void Print() { Console.WriteLine(this.ToString()); }

Correction: 'this' keyword cannot be used in a static method. FIX: Remove 'this.' or make method non-static.

OFFICIAL ANSWER KEY & GRADING SCHEME (QUESTIONS 1–100)

Section I MCQ Answer Keys:

Q1-10	1: B | 2: C | 3: C | 4: B | 5: B | 6: C | 7: A | 8: B | 9: B | 10: B
Q11-20	11: B | 12: B | 13: B | 14: C | 15: B | 16: C | 17: B | 18: A | 19: C | 20: B
Q21-30	21: B | 22: C | 23: B | 24: B | 25: B | 26: C | 27: B | 28: B | 29: C | 30: B

Grading Rubric Summary:

• **Part I (30 Marks):** 1 mark per correct MCQ choice.
• **Part II (20 Marks):** 1 mark per concise, accurate answer explaining core .NET memory/runtime concepts.
• **Part III (30 Marks):** 1 mark per valid C# code block demonstrating syntax and error prevention.
• **Part IV (20 Marks):** 1 mark per identified bug and correct code fix.
Passing Threshold: 50/100 Marks (50%) | **Honors Distinction:** 85/100 Marks (85%)